

Тема 9. СИНТЕЗ на ЦЕТНИ ИЗОБРАЖЕНИЯ, ОСВЕТЯВАНЕ

Основи на системите за компютърно виждане. Базови понятия. Светлина, цветовете, експониране, възпроизвеждане и композицията на кадъра.

Научните принципи на изобразяването (проекции, осветяване, сенки, прозрачност, цветове). Алгоритми и основни техники за синтез на двумерни цветни изображения. Разработване на интерактивни графични под-системи.

упр. 10 Основни техники и алгоритми за синтез на двумерни изображения. OpenGL – Изображения Цветни приложения.

9.1 СИСТЕМИ ЗА КОМПЮТЪРНО ЗРЕНИЕ

Главно средство за получаване на информация за околната среда е подсистемата за визуално възприемане. Задачата на **Системата за Компютърно Зрение** (СКЗ) от гледна точка на изкуствения интелект е възприемане на визуална информация за околната среда построяване на описание на тази среда и анализ на това описание с цел извличане на конкретна информация. Важността на СКЗ се потвърждава от факта, че човек получава по **зрителния канал** над **90%** от необходимата му информация за околната среда.

Под **компютърно зрение** се разбира: възприемане на визуална информация за околната среда с помощта на видео-сензор построяване на описание на сцената по нейното изображение; представяне на описанието на сцената в паметта на компютъра и неговата интерпретация с цел – разпознаване на обектите на основата на критерии, получени при обучението; изграждане на модел за околната среда.

СКЗ е съвкупност от специализирани устройства за формиране на **изображението** за околната среда; компютърна система, в която се **представя** и **съхранява** информацията от изображението; съответно програмно осигуряване. Програмното осигуряване се разработва на основата на методи и алгоритми за анализ, обучение и разпознаване на обектите от визуалната сцена и изграждането на модел на околната среда.

9.2 ФИЗИЧЕСКИ ОСНОВИ НА КОМПЮТЪРНОТО ЗРЕНИЕ

9.2.1 Видима светлина

Видимата светлина, възприемана посредством **зрението на човека**, посредством електромагнитни колебания с дължини на вълните от 400 до 700 nm ($1 \text{ nanometer} = 10^{-9}$, $1 \text{ micrometer} = 10^{-6}$). **Зрението** може да се разглежда като **възприемане** на отразена (или излъчваната от обектите в околната среда **електромагнитна енергия**, нейния анализ за извличане на информация за средата и за разпознаване на обектите в нея. Методите на компютърно зрение могат да се приложат за всяка област на електромагнитния спектър, където се получава изображение за наблюдаваната среда, започвайки от рентгеновото излъчване, ултравиолетовата, видимата, инфрачервената области и радарните изображения. Компютърното зрение представлява вид дистанционно наблюдение, т.е. измерване и анализ от разстояние [Гочев].

Областите с високо пропускане на електромагнитната енергия са: видимата ($0.4 \div 0.7 \mu\text{m}$), части от инфрачервената ($0.7 \div 100 \mu\text{m}$) и микровълновата области ($0.1 \div 30 \text{ cm}$) на електромагнитния спектър.

Човешкото зрение се осъществява във **видимата част** на електромагнитния спектър. Тази област се явява основна за системите за компютърно зрение, тъй като посредством тях се прави опит за алгоритмизиране на човешкото зрение.

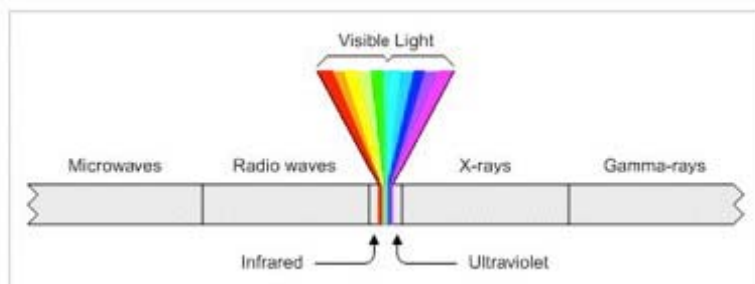
Светлината се дели ахроматична (лишена от цвят) и хроматична. Единственият атрибут на ахроматичната светлина е нейния интензитет. Понятието „ниво на сивото“ съответства на скаларното измерване на интензитета разпределено от черно през сиво до бяло. Хроматичната светлина обхваща спектъра на електромагнитна енергия от 400 до 700nm. Той се разделя в три ленти съответно от ляво на дясно: синя-Blue ($400 \div 500 \text{ nm}$), зелена – Green ($500 \div 600 \text{ nm}$), червена -Red ($600 \div 700 \text{ nm}$). Три са факторите при човешкото цветово усещане: цветови оттенък (Hue), наситеност (Saturation), яркост (Intensity). Оттенъкът определя името на цвета – червен, син и т.н. Наситеността определя чистотата на цвета.

Например, наситеността на червеното е по-висока от тази на розовото. Яркостта определя силата (интензитета) на цвета по същия начин, като в ахроматичните изображения. **Цветовото усещане** се основава на **дължината на светлинната вълна**. Международна комисия за осветление (International Commission of Illumination) определя като основни цветовете на единствените дължини на вълните на **700nm** (Red), **546.1nm** (Green) и **435.8nm** (Blue) [Гочев].

Дадена сцена може да се **цветово анализира** чрез използването на съответни филтри за червен, зелен и син цвят [Гочев]. Нека спектралния състав на цвета на дадена точка е $E(\lambda)$. Тогава съставните части на основните цветове се изчисляват посредством уравненията:

$$R = \int E(\lambda)R'(\lambda)d\lambda \quad G = \int E(\lambda)G'(\lambda)d\lambda \quad B = \int E(\lambda)B'(\lambda)d\lambda \quad (9.1)$$

където λ - дължина на вълната; $R'(\lambda)$, $G'(\lambda)$, $B(\lambda)$ са функции на разпределение.



Електромагнитен спектър

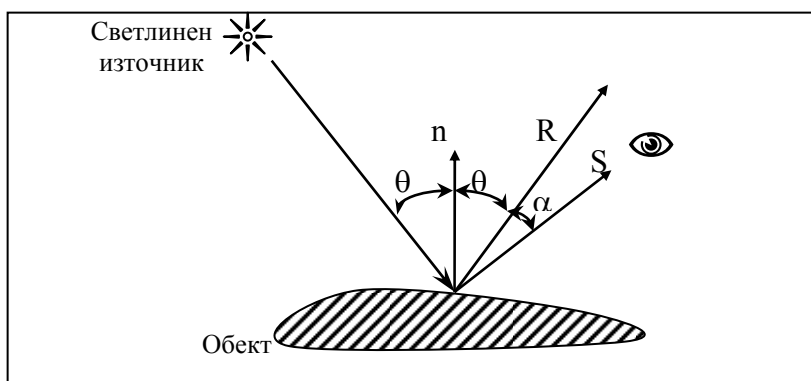
9.2.2 Модел на осветяване

Светлинната енергия, падаща на повърхността, може да бъде погълната, отразена или пропусната. Частично тя се поглъща и се превръща в топлина, частично се отразява или пропуска. Обектът се вижда, ако той отразява или пропуска светлина. Ако той поглъща цялата светлина, той е невидим и се нарича абсолютно черно тяло. Количеството погълната, отразена или пропусната светлина зависи от дължината на вълните. При осветяване с бяла светлина, при която интензивността на всички вълни е снижена еднакво, обектът изглежда сив. Ако се поглъща почти цялата светлина, обектът изглежда черен, а ако се поглъща съвсем малко светлина - бял. Ако се поглъщат само определени дължини на вълните, то светлината отразяваща се от обекта се изменя и обектът изглежда цветен.

Цветът се определя от погълнатите вълни с определена дължина.

Модел на осветяване

Прост модел на осветяване е представен на Фиг. 9.1



Свойствата на отразената светлина зависят от строежа и формата на източника на светлина и от ориентацията и свойствата на повърхността. **Отразената** от обекта светлина може да бъде **дифузна** и **огледална**. Дифузно е отражението на светлината, когато тя като че ли прониква под повърхността на обекта, поглъща се и отново се изпуска. Огледалното отражение става от външната повърхност на обекта. Едновременно върху обекта от реалната сцена пада и **разсеяна** светлина, разсеяна от окръжаващата среда.

Обединявайки компоненти на светлината се получава следния модел:

$$I = I_a \cdot k_a + \frac{I_l}{d+k} (k_d \cdot \cos \theta + w(i, \lambda) \cdot \cos^n \alpha) . \quad (9.2)$$

където: $w(i, \lambda)$ - крива на отражение = k_s (обикновено се заменя с константа k_s еднаква за трите компоненти); α - ъгъл на отражение; λ - дължина на вълната; n - степен по избор. k_a - коефициент на разсеяната светлина от 0 до 1. Коефициентът k_a е различен за трите основни цвята -син k_{ab} , червен k_{ar} и зелен k_{ag} , като определящ е цвета на източника на светлина. k_d - коефициент на дифузно отражение от 0 до 1. Коефициентът k_d е различен за трите основни цвята -син k_{ds} , червен k_{dr} и зелен k_{dg} , като определящ е цветът на повърхността.

I_a - интензивност на разсеяната светлина; I_l - интензивност на точковия източник;

θ - ъгъл между направлението на светлината и нормалата към повърхността; α - ъгъл на отражение; d - разстоянието от центъра на проекция до обекта; k - е произволна константа; n - степен по избор.

Ако запишем $\cos \theta = \frac{n \cdot \bar{L}}{|n| \cdot |L|} = \bar{n} \cdot \bar{L}$, където $\bar{n}, \bar{L}$ са единични вектори на нормалата към

повърхността и направлението към източника на светлина и $\cos \alpha = \frac{R \cdot \bar{S}}{|R| \cdot |\bar{S}|} = \bar{R} \cdot \bar{S}$,

където $\bar{R}, \bar{S}$ са единични вектори, определящи направлението на отразения лъч и вектора на наблюдателя, тогава моделът на осветяване с един източник на светлина ще има вида:

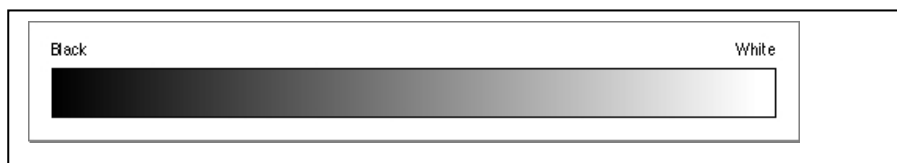
$$I = I_a \cdot k_a + \frac{I_l}{d+k} [k_d \cdot (\bar{n} \cdot \bar{L}) + k_s \cdot (\bar{R} \cdot \bar{S})] . \quad (9.3)$$

За цветно изображение се получат функциите на осветяване за всеки от трите основни цвята (червен, зелен и син).

9.3 ЦВЕТОВИ МОДЕЛИ

За представяне на цветовете в цифрови изображения се използват математически системи, наречени **цветови модели** или **цветови пространства** [Рачев]. Нито една от тези системи не е най-добрата. За различни цели се използват различни системи. Те са дефинира като едно-, две-, три- или четири мерно пространство, чиито компоненти, представляват интензивността на цветовете оттенъци, наситеност или яркост. Всички множества от цветове, които могат да се получат по пътя на смесването на основните цветове, представляват **цветно пространство** или **цветова гама**.

Цветовите модели могат да се разделят на две категории.: адитивни и субтрактивни. В адитивните модели новите цветове се получават по пътя на наслагването на основните цветове с различна интензивност и черния цвят. Колкото е по-голяма интензивността на добавения цвят, толкова по-близо е резултата до бялото. Бялото се получава при максимални интензивности на трите основни цвята, а компонентата на черното има минимално значение. **Адитивните модели** формират цвета в само светещите устройства като мониторите. В **субтрактивните** модели основния цвят се изважда от бялото. Колкото е по-голяма интензивността на изваждания цвят, толкова по-близък до черното е резултатът. Субтрактивните модели се използват при формиране на цветни изображения и отразяващи носители, например като хартия.



Фиг.9.2. Модел на сивата скала

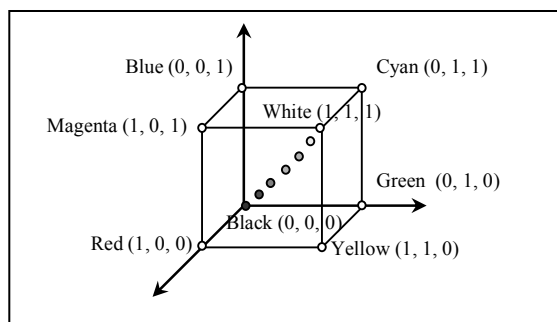
Най-известните цветови модели са: монохромен (чернобял) модел, модел на сивата скала, RGB модел, HSV модел, HLS модел, HSI модел. **Отделните модели са свързани помежду си с елементарни математически зависимости.**

9.3.1 Модел на сивата скала

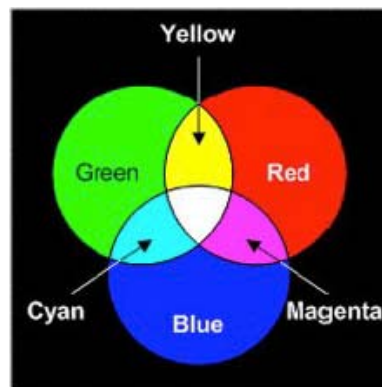
Обикновено това пространство има само един компонент, който се изменя от черно до бяло (Фиг.9.2). Използва се в чернобели монитори и принтери.

9.3.2 Модел RGB (Red-Green-Blue)

Моделът е адитивен и е най-често използван в графичните формати. представлява тримерното цветовото пространство на единичен куб, върховете на който съответстват точно на основните и допълнителни цветове.



Фиг. 9.3. Цветови Модел RGB

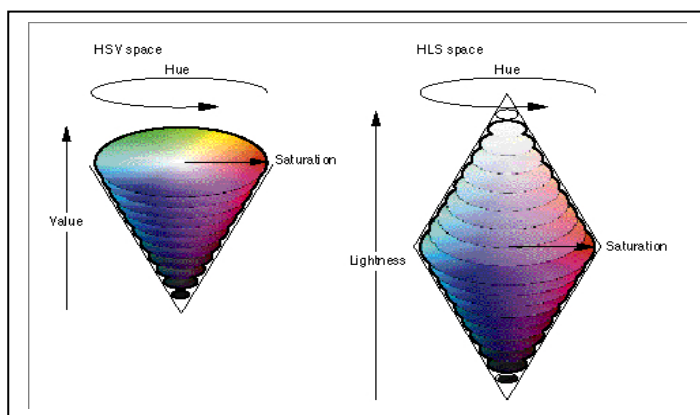


Основните цветове са – червен (Red), зелен (Green) и син, използвани във всички дисплеи. Диагоналната линия между началото на координатната система и точката на бялото представлява сивия цвят във всички негови нюанси (Фиг.9.3.). Всеки пиксел се представя във вида на трикомпонентна величина. На непрекъснатия диапазон от стойности 0 до 1 от цветовия модел се съпоставя диапазон от дискретни стойности, всяка в интервала 0÷255. Те се задават съответно за интензивността на трите компоненти : червено, зелено и синьо. Примери на цветове дефинирани чрез модела **RGB**:

- с кодове зададени в **десетична** бройна система са: RGB(127,127,127) – сиво, чиито три компоненти имат еднакви стойности , червено - RGB(255,0,0), бяло -RGB(255,255,255), жълто - RGB(255,255, 0), кафяво - RGB(255,200,100);; розово - RGB(255,102, 228).
- с **шестнадесетично** зададени кодове на компонентите като примерите: жълто - RGB(0xFF,0xFF,0x00); розово - RGB(0xFF,0x66, 0xEF).

Преобразуване на RGB в Gray Scale

- $W = (\max(R, G, B) + \min(R, G, B)) / 2$.
- $W = (R + G + B) / 3$.
- $W = 0.21 R + 0.71 G + 0.07 B$.



Фиг. 9.4. Цветови Модел HSV, HLS

9.3.3 Модел CMY (Cyan, Magenta, Yellow) е субтрактивен модел, който се използва за получаването на цветни изображения на бяла повърхност. При осветяване на изображения, получени с помощта на модела CMY, всеки от основните цветове поглъща допълнителния му. Синия поглъща червения, розовия поглъща зеления, жълтия – синия. Теоретически най-голямата интензивност на всичките три основни цвята на субтрактивния модел трябва да постигне черния цвят. На практика, това не става и затова в модела добавят и четвърти компонент – Black (черно), обозначен с K в модела **Модел CMYK** (Cyan, Magenta, Yellow, Black).

Тъй като RGB е **пълно цветови** модел то всички други **пълни цветови модели могат да се получат от него чрез трансформиране, въобще нелинейно**. Инверсията на RGB е моделът CMY, прилаган в принтерите.

Други известни модели са: **Модел HSV** (Hue, Saturation, Value), **Модел HLS** (Hue, Luminance, Saturation), **Модел HSI** (Hue, Saturation, Intensity) (Фиг. 9.4.).

Варианти на дължината на описанието на цвета на засветяване на всеки пиксел: 1 бит/точка за монохромни (чернобели) изображения; 4 бита/точка за изображение от 16 цвята; 8 бита, 1 байт/точка за изображение от 256 цвята; 2 байта/точка за ≈ 65000 цвята и 4 байта/точка за $\approx 4 \cdot 10^6$ цвята, 3 байта на пиксел в случая на RGB цветовия модел.

Използването на цветовете в обработката на изображенията често опростява извличането на информация за **идентификация на обектите** в него. Дадена сцена може да се цветово анализира чрез използването на съответни филтри за червен, зелен и син цвят. Нека спектралния състав на цвета на дадена точка е $E(\lambda)$. Тогава съставните части на основните цветове се изчисляват посредством уравненията (9.1), където $R'(\lambda)$, $G'(\lambda)$, $B(\lambda)$ са функции на разпределение.

9.4 OPENGL – СИНТЕЗИРАНЕ НА ЦВЕТНИ ИЗОБРАЖЕНИЯ

Определяне на режима на представяне с функцията **glutInitDisplayMode**, която приема няколко аргумента разделени със символа "|". Това са :

- **GLUT_SINGLE** или **GLUT_DOUBLE** за единичен или двоен буфер. За предпочитане е използването на двоен буфер, освен ако няма да се представя някоя 2D фигура. За триизмерна графика е задължително използването на двоен буфер, който е и много по-бърз. При GLUT_DOUBLE само единия буфер е видим, а в другия се създава графиката и след това двата буфера се разменят.

- **GLUT_DEPTH**, **GLUT_STENCIL** и/или **GLUT_ACCUM**. Първата константа определя наличието на z-буфер. За триизмерна графика неговото включване е задължително.

- **GLUT_RGBA** или **GLUT_INDEX** определя начина за изграждане на цветовете във вашата програма. При GLUT_INDEX цвета се определя от т.н. цвятова палитра, която може да съдържа 16 или 256 цвята и на всяко число от 0 до 15 или от 0 до 255 съответства точно определен цвят - стандарт, който възпрепятства използването на други ефекти като например - динамична светлина. За предпочитане е използването на **GLUT_RGBA**, при който цвета се сформира от количеството на RED, GREEN, BLUE цветовете и ALPHA канала (полу-прозрачност).

✓ Определянето на цвят в **RGBA** режим става с функцията:

glColor{34}f(v)(R, G, B, (A)) - цвета се изгражда в зависимост от наситеността на червения, синия и зеления цвят, която се определя в части от цялото. Ако искате наполовина наситено червено трябва да извикате функцията със следните параметри :

glColor3f(0.5 , 0.0 , 0.0);

Четвъртият аргумент, който не е задължителен е прозрачността. Тя обаче трябва да се включи предварително с други функции. Полу-прозрачността се разглежда в лекция 8, а стойност 1.0 , показва че няма полу-прозрачност.

✓ При **Color-Index** режим функцията за определяне на цвят е :

glIndex(index) - като число от 0 - 255 определя различен цвят.

Можете да определите различен цвят за всеки връх и така вашата фигура ще прелива в зависимост от цвета на нейните върхове.

- ✓ С функцията **glShadeModel(GLenum mode)** може да определите начина на определяне на цвета. Тя може да приеме една от константите:
 - **GL_SMOOTH** - за преливащ цвят, което е по-подразбиране.
 - **GL_FLAT** - за едноцветни фигури, независимо от цвета на върховете
- ✓ С функцията **glClearColor()**, може да се запълни цветовия буфер с определен цвят или - да се определи цвета на фона. Нейният прототип е:

void glClearColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha) - тя задава RGBA цвета на фона. Тази функция се използва и за създаване на реалистична мъгла.

9.5 OPENGL - ИЗОБРАЖЕНИЯ

Създаването на реалистична 3D графика е немислимо без използването на готови изображения, които служат за създаването на текстури. За да се използва изображение в OpenGL, то трябва да се представи в масив от RGB стойности. OpenGL не поддържа нито един готов формат затова библиотеката GLAUX предоставя готова функция която може да извърши конвертирането от *.bmp файл :

AUX_RGBImageRec* auxDIBImageLoad("име.bmp") - тя приема името на *.bmp файла и връща указател към структурна променлива от тип **AUX_RGBImageRec**. Структурата **AUX_RGBImageRec** е дефинирана в библиотеката **GLAUX** и съдържа няколко члена. Това са **sizeX** , **sizeY** от тип **int**, в които се запазват съответно ширината и дължината на зареденото изображение и ***data** от тип **unsigned char**, където се запазват RGB стойностите на изображението.

Пример за зареждане изображението wood.bmp :

```
AUX_RGBImageRec* image;  
image = auxDIBImageLoad("wood.bmp");
```

Позицията на изображението в прозореца се задава с неговата позиция по x, y и/или z. Това става най-лесно със следните варианти на функцията **glRasterPos*()** :

glRasterPos2f(GLfloat x, GLfloat y) - приема x и y позицията на изображението.

glRasterPos3f(GLfloat x, GLfloat y, GLfloat z) - приема x, y и z позицията на изображението.

Трансформациите на **MODELVIEW** и **RPROJECTION** матрицата се отразяват на координатите на изображението. Освен това е препоръчително да се използват ортогографична перспектива, когато се рисуват изображения. След зареждането на изображението, то може да се покаже на екрана. Това става с функцията :

void glDrawPixels(GLsizei width, GLsizei height, GLenum format, GLenum type, const GLvoid *pixels) - тя приема 5 аргумента. Първият и вторият са ширината и височината на изображението. Те се намират съответно в **image->sizeX** и **image->sizeY**. Третият обозначава цветовия режим. В случая това е **GL_RGB**. Четвъртият аргумент указва типа на данните в масива с RGB стойностите, в случая това ще **GL_UNSIGNED_BYTE**. Последният аргумент трябва да е името на масива с RGB стойностите - **image->data**. Ето как ще изглежда функцията след като се подадат нужните аргументи :

```
glDrawPixels( image->sizeX, image->sizeY, GL_RGB, GL_UNSIGNED_BYTE, image->data );
```

OpenGL предлага още няколко полезни функции за манипулация на изображенията, преди тяхното изрисване. Едната от тях е :

void glPixelTransfer{fi}(GLenum pname, GLfloat param) - тя приема 2 аргумента. С тази функция може да се увеличи или намали един от трите цветови компонента (**RGB**) на изображението. За първи аргумент се задава **GL_RED_SCALE**, **GL_GREEN_SCALE** или **GL_BLUE_SCALE**. Втори аргумент е число(наситеност), което ще се умножи (насити) избрания цветови компонент на изображението. Числа по-малки от 1.0 водят до намаляване количеството на избрания цвят, а числа по-големи от 1.0 водят до неговото увеличаване.

Функция за мащабиране на изображението е **glPixelZoom()**, и нейния прототип:

void glPixelZoom(GLfloat xfactor, GLfloat yfactor) - функцията приема два аргумента, които определят мащабирането по x и y. Числа по-големи от 1.0 водят до увеличаване на изображението, и обратно - по-малки от 1.0 намаляват изображението. **Ако се зададе отрицателно число ще се получи огледално изображение.**

Функция за четене на част от екрана и съхраняване на изображението в масив с прототип:

void glReadPixels(GLint x, GLint y, GLsizei width, GLsizei height, GLenum format, GLenum type, GLvoid *pixels) - функцията приема 7 аргумента. Първите два определят позицията на долния ляв ъгъл на правоъгълника, който трябва да се прочете, следващите два аргумента - неговата ширина и дължина. Петият аргумент определя какво точно да бъде прочетено. Може да бъде :

GL_RGB - RGB цвета

GL_RGBA - RGBA цвета (RGB цвета и Alpha канала)

GL_COLOR_INDEX - индекса на цвета, ако се използва **COLOR_INDEX** режим

GL_RED - червения цветови компонент

GL_GREEN - зеления цветови компонент

GL_BLUE - синия цветови компонент

За шести аргумент трябва да се избере типа на данните, които ще се запишат в масива. Може да бъде - **GL_UNSIGNED_BYTE**, **GL_BYTE**, **GL_BITMAP**, **GL_UNSIGNED_SHORT**, **GL_SHORT**, **GL_UNSIGNED_INT**, **GL_INT** или **GL_FLOAT**. За последен аргумент трябва да се подаде името на масива, в който ще се съхрани изображението.

9.6 OPENGL - ПОЛУПРОЗРАЧНОСТ

В примерната графика голяма част от ефектите са реализирани благодарение на полу-прозрачността. Четвъртият компонент от **RGBA** режима се използва от **OpenGL** за установяване на степен на прозрачност. За да активирате обаче т.н. **Alpha Blending** не е достатъчно само да се зададе четвъртият аргумент във функцията **glColor4f**. Има някои важни особености.

Най-напред трябва да се активира полу-прозрачността с функцията **glEnable()** с аргумент **GL_BLEND**. След това трябва да се определи начина, по който се смесват цветовете на предния и задния обект, в което всъщност се състои полу-прозрачността. Формулата по която се изчислява полу-прозрачността в **OpenGL**:

(Rs*Sr+Rd*Dr, Gs*Sg+Gd*Dg, Bs*Sb+Bd*Db, As*Sa+Ad*Da) , където **Rs,Gs,Bs** и **As** са **RGBA** стойностите на предния обект, които задават с **glColor4f()** , а **Rd,Gd,Bd** и **Ad** съответно **RGBA** стойностите на задния обект.

Sr,Sg,Sb,Sa и **Dr,Dg,Db,Da** са т.н. фактори на полу-прозрачността, от които зависи как ще се смесят цветовете на двата обекта, за да се постигне полу-прозрачност. Благодарение на тях може да се възпроизведе, най-различни ефекти на полу-прозрачност. Първата поредица от четири аргумента се отнася за предния обект, а втората за задния обект. Това се извършва с функцията:

void glBlendFunc (GLenum sfactor, GLenum dfactor), която приема два аргумента, от които зависи какви стойности ще получат факторите на полу-прозрачност съответно за предния и задния обект. Възможните аргументи са:

GL_ZERO - цвета на обекта не се взема предвид. За стойности на фактора на полу-прозрачност, на който и да е от двата обекта се предават стойностите (0, 0, 0, 0). Може да се използва както за преден, така и за заден обект.

GL_ONE - използва се цвета на обекта. За стойности на фактора на полу-прозрачност, на който и да е от двата обекта се предават стойностите (1, 1, 1, 1). Също няма значение за кой от двата обекта определят фактор на полу-прозрачност.

GL_DST_COLOR - цвета на обекта се умножава по този на задния обект. Може да се използва единствено за предния обект, т.е. само като първи аргумент на функцията. За фактор на полу-прозрачност на предния обект се предават стойностите **Rd,Gd,Bd** и **Ad** т.е. **RGBA** цвета на задния обект.

GL_SRC_COLOR - цвета на обекта се умножава по този на предния обект. Може да се използва единствено за задния обект, т.е. само като втори аргумент на функцията. За

фактор на полу-прозрачност на задния обект се предават стойностите **Rs,Gs,Bs** и **As** т.е. **RGBA** цвета на предния обект.

GL_ONE_MINUS_DST_COLOR - цвета на обекта се умножава с (1, 1, 1, 1 минус цвета на задния обект). Може да се използва само за предния обект (**sfactor**) и реално фактора за полу-прозрачност на предния обект получава стойности (1, 1, 1, 1) - (**Rd, Gd, Bd, Ad**), където последните четири както виждате представляват цвета на задния обект.

GL_ONE_MINUS_SRC_ALPHA - цвета на обекта се умножава с (1, 1, 1, 1 минус цвета на предния обект). Може да се използва само за аргумент на **dfactor**. Съответно тук за фактор на полу-прозрачност задния обект получава стойностите (1, 1, 1, 1) - (**Rs, Gs, Bs, As**).

GL_SRC_ALPHA - цвета на който и да е от двата обекта(може да го използвате както за **sfactor** така и за **dfactor**) се умножава с четвъртата стойност (**Alpha**) на предния обект стойност т.е. за фактори на полу-прозрачност обекта получава стойностите (**As, As, As, As**).

GL_DST_ALPHA - абсолютно същото с изключение на това, че за фактори на полу-прозрачност обекта получава четвъртата **RGBA** стойност от цвета на задния обект - (**Ad, Ad, Ad, Ad**).

GL_ONE_MINUS_DST_ALPHA - цвета на обекта се умножава с (1, 1, 1, 1 минус четвъртата **RGBA** стойност на задния обект) т.е. за фактор на полу-прозрачност се получават стойностите (1, 1, 1, 1) - (**Ad, Ad, Ad, Ad**) Този аргумент може да бъде използван във функцията **glBlendFunc()** както за предния, така и за задния обект.

GL_ONE_MINUS_SRC_ALPHA - същото като по горе с изключение на това че се използва **alpha** от предния обект т.е. за фактор на полу-прозрачност се приемат стойностите (1, 1, 1, 1) - (**As, As, As, As**).

Най-често използваните фактори за полу-прозрачност са **GL_SRC_ALPHA** за предния обект и **GL_ONE_MINUS_SRC_ALPHA** за задния обект. Важно е да се отбележи че ако се използва полу-прозрачност трябва да се изрисува първо задния обект и след това този отпред (за който е разрешена полу-прозрачността

Технология Antialiasing

Тя предполага,че при изобразяването на линия се използват по-голям брой пиксели като различното е, че за крайните пикселни единици се задава полу-прозрачност. Така окото ни не може да разграничи ясно чупките на линията и ефекта се получава - имаме гладка, не-нащърбена линия. Пример как да се изглаждат точки (изглеждат "по-кръгли"), прости линии или обекти изобразени с помощта на множество линии или точки. Изглаждането на краищата на запълнени обекти е доста по-трудно и няма да бъде разгледано, а и вместо него може да бъде използвано пълно-екранно изглаждане на всичко по сцената. За да се изгладят линиите или да се направят точките по-заоблени, е нужно да подадете на функцията **glEnable()** за аргумент **GL_LINE_SMOOTH** или **GL_POINT_SMOOTH**. След това е нужно да се разреши полу-прозрачност, като се подадете за аргумент на същата функция константата **GL_BLEND** и след това да се определи режим на смесване на цветовете на обектите, като се подаде на функцията **glBlendFunc()** аргументите **GL_SRC_ALPHA** и **GL_ONE_MINUS_SRC_ALPHA**, съответно за фактори на полу-прозрачност на предния и задния обект.